



Universidad
Indoamérica

UNIT 1 · INFORMATION TECHNOLOGY ENGINEERING · 4TH SEMESTER

Fundamentals of Operating Systems

Andrés Rubio Proaño

Universidad Tecnológica Indoamérica · School of Engineering

What is an operating system?

What we will do

- General principles
- Practical programming examples (9 labs in C)
- Study the lowest layers of software
- **New:** how operating systems host AI agents

What it is *not*

- Not a C course → read *The C Programming Language* (Kernighan and Ritchie)
- Not a shell course → learn your shell: use `man`

Nine units, one lab per unit

1	Introduction	TD0	6	Signals and shared memory	TD5
2	System calls	TD1	7	Threads and concurrency	TD6
3	Files	TD2	8	Networking: sockets	TD7
4	Processes and memory	TD3	9	AI-native OS	TD8
5	Pipes	TD4			

An operating system...

- **manages and hides** hardware complexity from the programmer
- provides the interface of a **virtual machine**: processes, files, virtual memory, sockets
- **protects** users and programs from each other

The services of the operating system

Resource manager

- Hides hardware complexity
- Resolves access conflicts and shares fairly
- Handles errors
- Prevents misuse of the machine

Makes the machine easy to use

- Standard libraries (glibc, graphics, network)
- Programming becomes simpler and portable (POSIX)

Which resources? Which services?

CPU time

Scheduling tasks on the cores (U4)

Memory

Allocation and protection (U4, U6)

Input/output

Files and permissions, network, windows (U2, U3, U8)

Energy

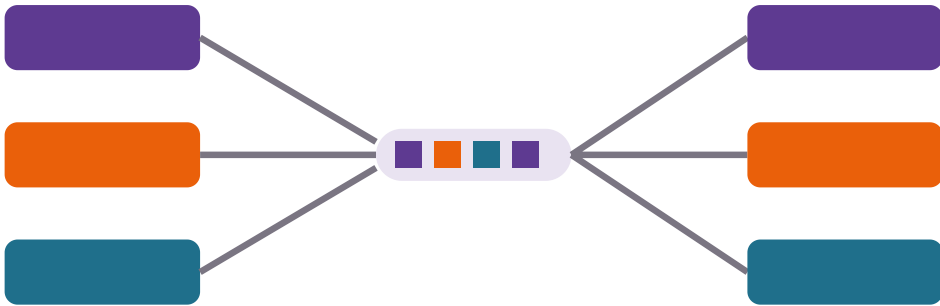
CPU frequency, suspend

Accelerators (2026)

GPUs and NPUs for AI: the OS shares them like any resource (U9)

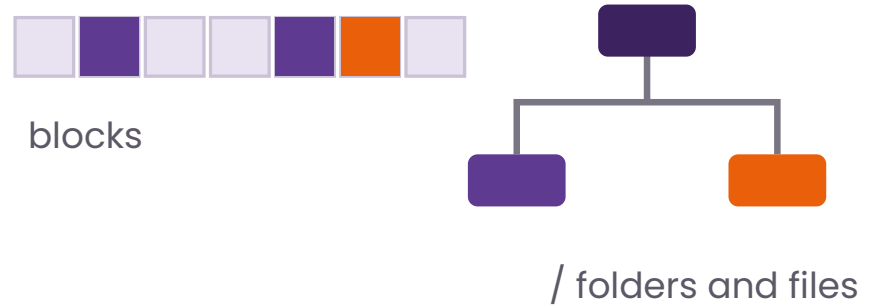
Structuring data

Multiplexing the network



A single bitstream with data from many programs to many servers, separated on arrival.

Organising the disk



A huge array of blocks storing a tree of folders and files (U3).

In supercomputers (TOP500, June 2026)

#	SYSTEM	SITE	HPL
1	LineShine	Shenzhen, China	2,198 EF
2	El Capitan	LLNL, USA	1,809 EF
3	Frontier	ORNL, USA	1,353 EF
4	Aurora	ANL, USA	1,012 EF
5	JUPITER Booster	Jülich, Germany	1,000 EF

1 EF = 10^{18} floating-point operations per second. Five systems already exceed one exaflop. LineShine runs Kylin OS, a Linux-based distribution. Source: top500.org (the list is updated every 6 months).

In data centres and in the cloud

Data centres

Banks, insurers, universities: thousands of servers, almost all running Linux, many virtualised.

Cloud: no physical location

Virtual machines and containers spread around the world; your "computer's" OS is a service (storage, mail, compute).

In mobile, embedded and critical systems

- Phones: Android (Linux kernel) and iOS (XNU kernel)
- Embedded: modems, TVs, cars; real-time OSes such as FreeRTOS, Zephyr, QNX or VxWorks

Critical systems: sometimes without a general-purpose OS

- A single program that does not need to run elsewhere
- Real time: a specific OS for a set of tasks and events, or none
- Synchronous languages that guarantee reaction times: Lustre, Esterel

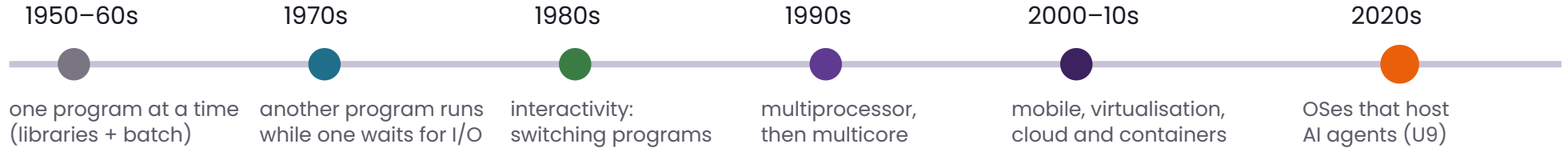
Hiding hardware complexity

Machines are very different:

- type and number of processors and cores (this machine: 28)
- amount of memory
- type of disk or network card

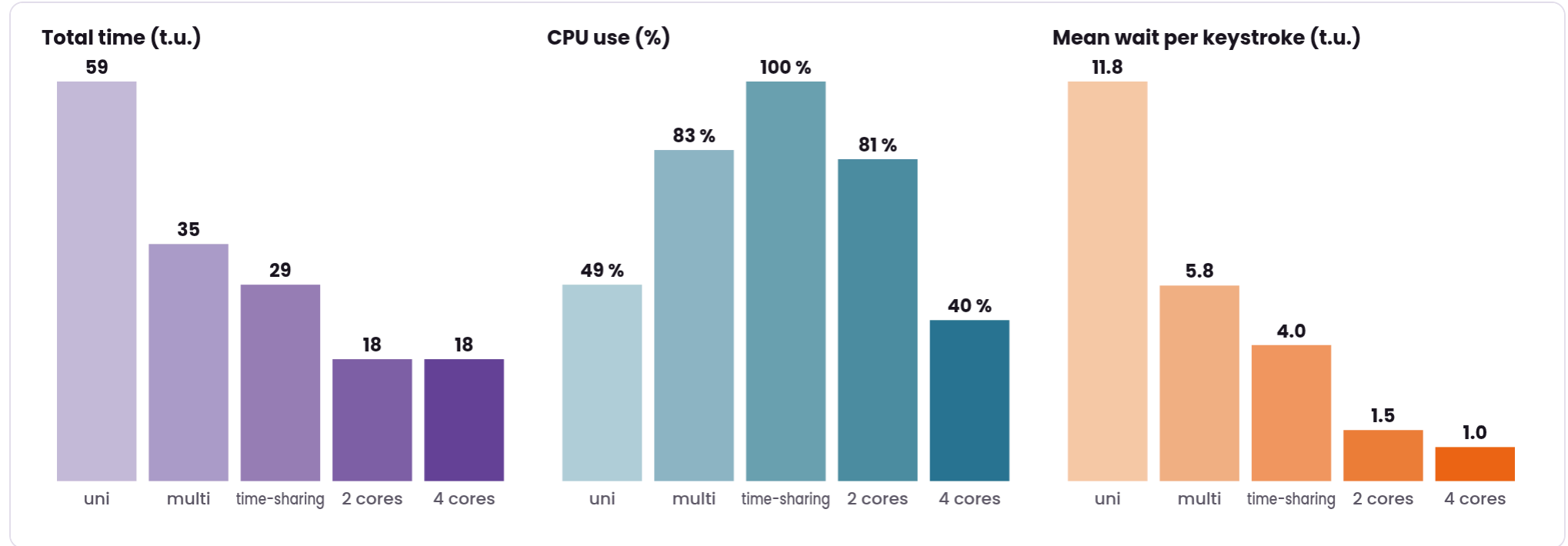
We want **the same program to work on all of them** without changes: with more or less memory, with any network card, with a local SSD or a remote NAS.

A bit of history

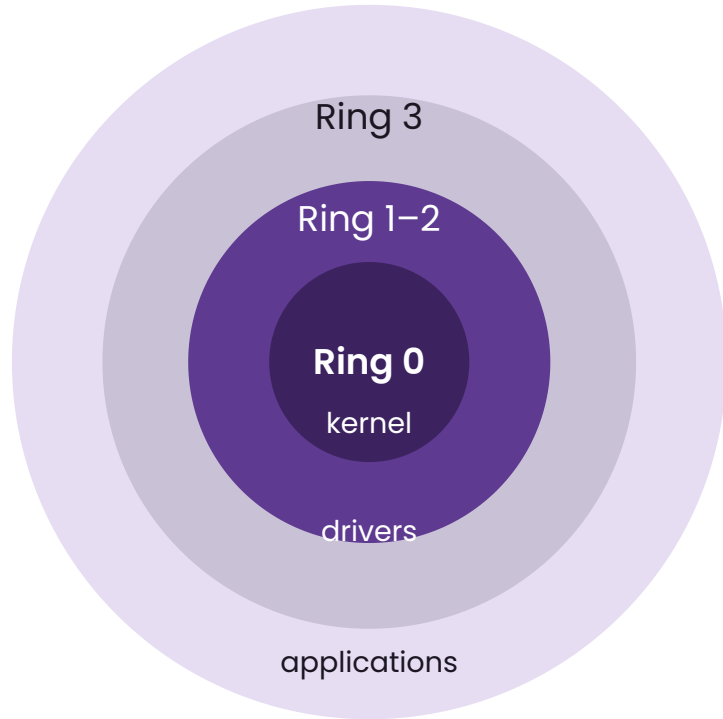


At first an OS was just a few libraries, with two assumptions: *no malicious programmers* and *one program at a time*. Consequence: wasted machine and user time. Every era removed an assumption... and added a problem this course studies.

Wasted machine time and user time



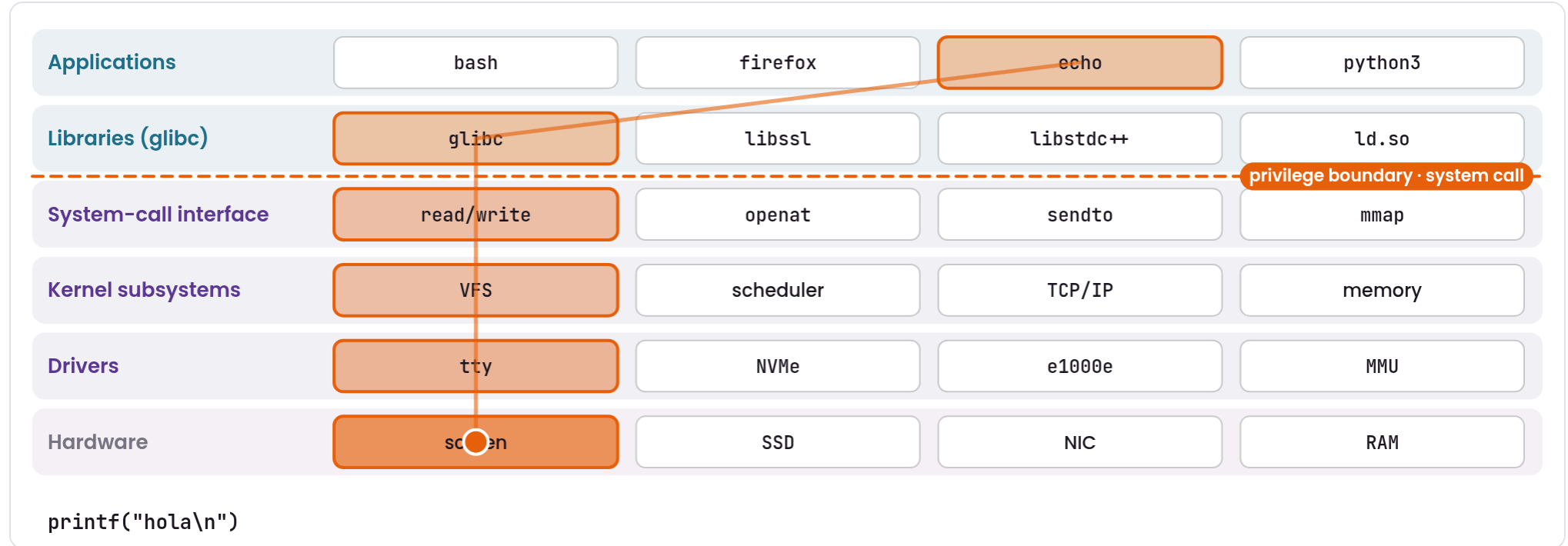
Structure of a modern OS: an onion



x86 defines 4 rings; Linux and Windows use only two: **ring 0** (kernel, most privileged) and **ring 3** (applications, least privileged).

The kernel **is not root**: root is a user; the kernel is the code running in ring 0.

The components, travelled by a request

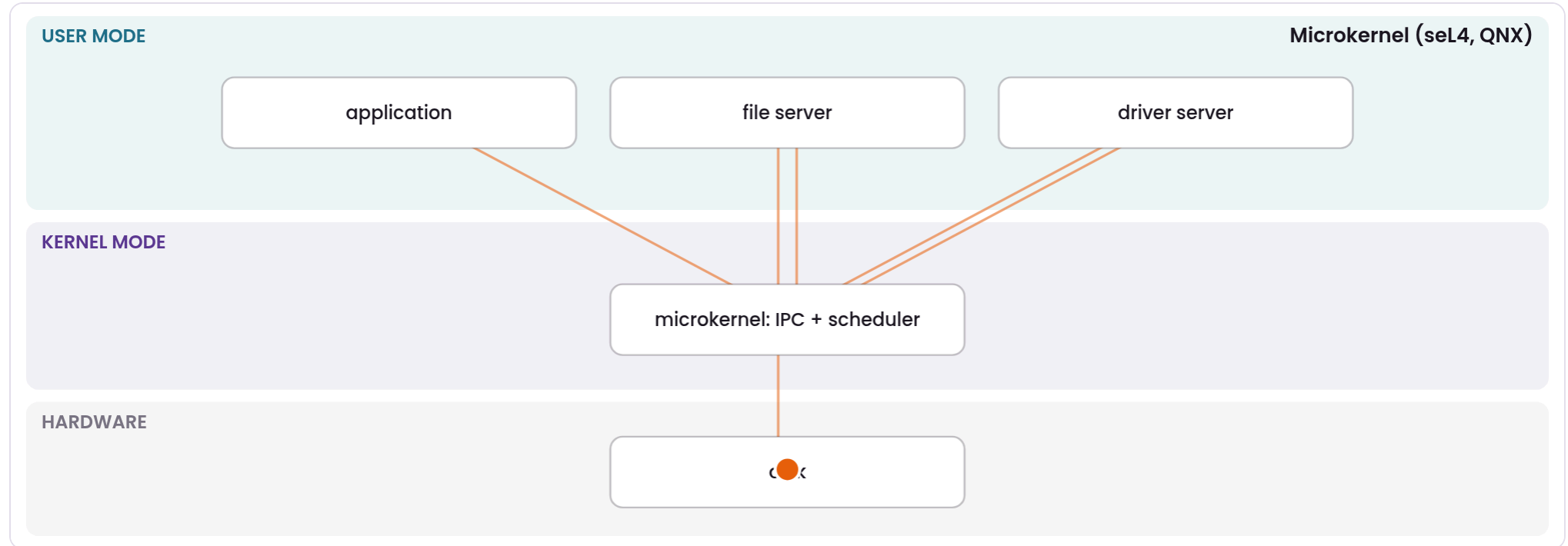


A very complex piece of software

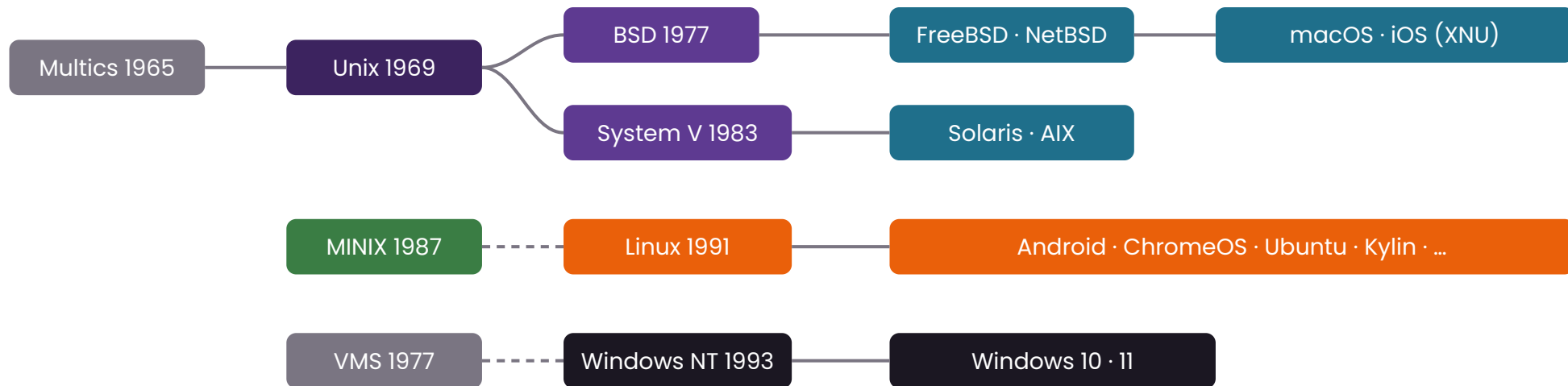
- The Linux kernel has tens of millions of lines of code; most of them are **drivers**.
- Thousands of developers per release; a new release every 9–10 weeks.
- On the course machine: Linux 7.0 and **258 loaded modules**.

That is why architecture matters: what happens if one of those drivers fails?

Monolithic, microkernel, hybrid

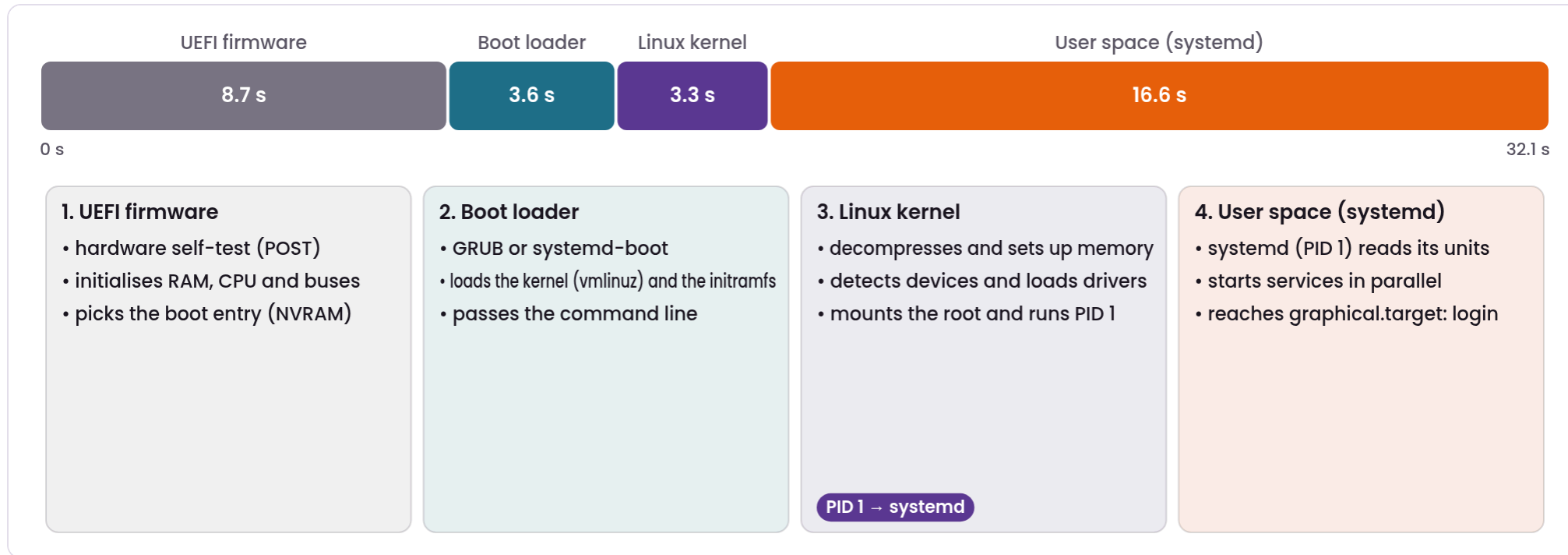


Examples and genealogy



Solid line: code descent; dotted: design influence (Linux was inspired by MINIX; Windows NT by VMS, with the same lead architect). Also: GNU/Hurd (microkernel), Plan 9, VxWorks.

From power-on to login (real times)



Further reading

- Silberschatz, Galvin, Gagne. *Operating System Concepts*, 10th ed.
- Arpaci-Dusseau. *Operating Systems: Three Easy Pieces* (free)
- Online courses: CS 162 (Berkeley), CS 140 (Stanford)
- top500.org · man7.org/linux/man-pages · docs.kernel.org



Universidad
Indoamérica

TAKE-AWAYS · LAB TD0

Three questions

1. Which services does an OS give as a resource manager?
2. Why is the kernel not the same as root?
3. What does a microkernel gain and lose compared to Linux?